

Compiler Design

Lab Assessment 3

Student Name: Avyakth Shriram

Register Number: 24BCE5461

PROGRAM 1

shift_reduce_parser.c

Aim:

To design, implement, and test a bottom-up Shift-Reduce Parser in C for a given Context-Free Grammar (CFG) to validate whether an input string belongs to the language defined by the grammar.

About:

Shift-Reduce is a bottom-up parsing technique that reads input tokens from left to right, pushing them onto a stack (**Shift**) until a substring matches a production rule's right-hand side, replacing it with the corresponding non-terminal (**Reduce**).

Procedure:

1. **Initialize Data Structures:** Create a character array to act as the parser stack (initialized empty with $\text{top} = -1$) and read the input string, appending an end-marker token (\$).
2. **Read & Shift:** Read tokens from the input buffer from left to right and push them onto the stack one by one.
3. **Check for Handle (Reduction):** After every shift, inspect the top elements of the stack to see if they match the right-hand side (RHS) of any production rule ($E \rightarrow \text{id}$, $E \rightarrow E + E$, $E \rightarrow E * E$, $E \rightarrow (E)$).
4. **Reduce:** If a match (handle) is found, pop the matching RHS symbols from the stack and push the corresponding left-hand side (LHS) non-

terminal symbol (E). Repeat this step greedily until no further reductions are possible on the current stack top.

5. Evaluate Acceptance/Rejection:

- a. **Accept:** If the stack contains only the start symbol E and the input pointer reaches the end-marker $\$,$ print the parsing steps and report success.
- b. **Reject:** If no further shifts or reductions are possible and the stack does not reduce to E at $\$,$ or an invalid token sequence occurs, terminate and flag a syntax error.

C Program Code:

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

char stack[100];
int top = -1;
char input[100];
int ip = 0;

void push(char c) {
    stack[++top] = c;
    stack[top + 1] = '\0';
}

void pop() {
    if (top >= 0) {
        stack[top] = '\0';
        top--;
    }
}

void print_step(const char *action) {
    printf("%-20s %-20s %-25s\n", stack, &input[ip], action);
}

// Checks stack top for reducible patterns: id, E+E, E*E, (E)
```

```

int reduce() {
    // Check for "id"
    if (top >= 1 && stack[top - 1] == 'i' && stack[top] == 'd') {
        pop(); // remove 'd'
        pop(); // remove 'i'
        push('E');
        print_step("Reduce: E -> id");
        return 1;
    }
    // Check for "E+E"
    if (top >= 2 && stack[top - 2] == 'E' && stack[top - 1] == '+' && stack[top] == 'E')
    {
        pop(); // remove 'E'
        pop(); // remove '+'
        pop(); // remove 'E'
        push('E');
        print_step("Reduce: E -> E+E");
        return 1;
    }
    // Check for "E*E"
    if (top >= 2 && stack[top - 2] == 'E' && stack[top - 1] == '*' && stack[top] == 'E')
    {
        pop(); // remove 'E'
        pop(); // remove '*'
        pop(); // remove 'E'
        push('E');
        print_step("Reduce: E -> E*E");
        return 1;
    }
    // Check for "(E)"
    if (top >= 2 && stack[top - 2] == '(' && stack[top - 1] == 'E' && stack[top] == ')') {
        pop(); // remove ')'
        pop(); // remove 'E'
        pop(); // remove '('
        push('E');
        print_step("Reduce: E -> (E)");
        return 1;
    }
    return 0;
}

int main() {
    char raw_input[100];

```

```

printf("=====\\n");
printf("    EXPERIMENT 5: SHIFT-REDUCE PARSER    \\n");
printf(" Grammar:                \\n");
printf("  1. E -> E + E            \\n");
printf("  2. E -> E * E            \\n");
printf("  3. E -> (E)              \\n");
printf("  4. E -> id              \\n");

printf("=====\\n");
printf("Enter input string (e.g., id+id*id, (id+id), id*id): ");
if (scanf("%99s", raw_input) != 1) return 0;

// Append end-marker '$'
snprintf(input, sizeof(input), "%s$", raw_input);

stack[0] = '\\0';
top = -1;
ip = 0;

printf("\\n%-20s %-20s %-25s\\n", "STACK", "INPUT BUFFER", "ACTION");
printf("-----\\n");

while (1) {
    // Shift step if not at end of input
    if (input[ip] != '$') {
        if (input[ip] == 'i' && input[ip + 1] == 'd') {
            push(input[ip++]);
            push(input[ip++]);
            print_step("Shift (id)");
        } else {
            char ch = input[ip++];
            push(ch);
            char action[30];
            snprintf(action, sizeof(action), "Shift (%c)", ch);
            print_step(action);
        }
    }

    // Apply reductions greedily
    while (reduce());

    // Acceptance condition: Stack has only 'E' and input is at '$'

```

```
    if (top == 0 && stack[0] == 'E' && input[ip] == '$') {  
        printf("%-20s %-20s %-25s\n", stack, "$", "ACCEPT");  
        printf("\n>> SUCCESS: The string \"%s\" is VALID and ACCEPTED.\n",  
raw_input);  
        break;  
    }  
  
    // Error / Rejection condition  
    if (input[ip] == '$' && !(top == 0 && stack[0] == 'E')) {  
        printf("\n>> ERROR: The string \"%s\" is INVALID and REJECTED.\n",  
raw_input);  
        break;  
    }  
}  
  
return 0;  
}
```

Screenshots & Output:

```
avyakths@AvysLegionSlim5:~$ cd Compiler-Design-Lab/
avyakths@AvysLegionSlim5:~/Compiler-Design-Lab$ ls
Lab_Assignment_1 Lab_Assignment_2 Lab_Assignment_3
avyakths@AvysLegionSlim5:~/Compiler-Design-Lab$ cd Lab_Assignment_3
avyakths@AvysLegionSlim5:~/Compiler-Design-Lab/Lab_Assignment_3$ ls
lr0_parser lr0_parser.c shift_reduce_parser shift_reduce_parser.c
avyakths@AvysLegionSlim5:~/Compiler-Design-Lab/Lab_Assignment_3$ ./shift_reduce_parser
=====
EXPERIMENT 5: SHIFT-REDUCE PARSER
Grammar:
1. E -> E + E
2. E -> E * E
3. E -> (E)
4. E -> id
=====
Enter input string (e.g., id+id*id, (id+id), id*id): id

STACK      INPUT BUFFER      ACTION
-----
id          $              Shift (id)
E           $              Reduce: E -> id
E           $              ACCEPT

>> SUCCESS: The string "id" is VALID and ACCEPTED.
avyakths@AvysLegionSlim5:~/Compiler-Design-Lab/Lab_Assignment_3$ ./shift_reduce_parser
=====
EXPERIMENT 5: SHIFT-REDUCE PARSER
Grammar:
1. E -> E + E
2. E -> E * E
3. E -> (E)
4. E -> id
=====
Enter input string (e.g., id+id*id, (id+id), id*id): id+id*id

STACK      INPUT BUFFER      ACTION
-----
id          +id*id$          Shift (id)
E           +id*id$          Reduce: E -> id
E+          id*id$           Shift (+)
E+id        *id$           Shift (id)
E+E         *id$           Reduce: E -> id
E           *id$           Reduce: E -> E+E
E*          id$           Shift (*)
E*id        $           Shift (id)
E*E         $           Reduce: E -> id
E           $           Reduce: E -> E*E
E           $           ACCEPT

>> SUCCESS: The string "id+id*id" is VALID and ACCEPTED.
```

```

avyakths@AvysLegionSlim5:~ × + v
avyakths@AvysLegionSlim5:~/Compiler-Design-Lab/Lab_Assignment_3$ ./shift_reduce_parser
=====
EXPERIMENT 5: SHIFT-REDUCE PARSER
Grammar:
1. E -> E + E
2. E -> E * E
3. E -> (E)
4. E -> id
=====
Enter input string (e.g., id+id*id, (id+id), id*id): (id+id)*id

STACK          INPUT BUFFER          ACTION
-----
(              id+id)*id$          Shift ( (
(id            +id)*id$           Shift ( id
(E            +id)*id$           Reduce: E -> id
(E+          id)*id$             Shift ( +
(E+id        )*id$              Shift ( id
(E+E        )*id$              Reduce: E -> id
(E          )*id$              Reduce: E -> E+E
(E          *id$              Shift ( )
E          *id$              Reduce: E -> (E)
E*         id$              Shift ( *
E*id        $              Shift ( id
E*E         $              Reduce: E -> id
E          $              Reduce: E -> E*E
E          $              ACCEPT

>> SUCCESS: The string "(id+id)*id" is VALID and ACCEPTED.
avyakths@AvysLegionSlim5:~/Compiler-Design-Lab/Lab_Assignment_3$ ./shift_reduce_parser
=====
EXPERIMENT 5: SHIFT-REDUCE PARSER
Grammar:
1. E -> E + E
2. E -> E * E
3. E -> (E)
4. E -> id
=====
Enter input string (e.g., id+id*id, (id+id), id*id): ((id+id)*(id+id))+id

STACK          INPUT BUFFER          ACTION
-----
(              (id+id)*(id+id))+id$ Shift ( (
((            id+id)*(id+id))+id$ Shift ( (
((id          +id)*(id+id))+id$ Shift ( id
((E          +id)*(id+id))+id$ Reduce: E -> id
((E+        id)*(id+id))+id$ Shift ( +
((E+id      )*(id+id))+id$ Shift ( id
((E+E      )*(id+id))+id$ Reduce: E -> id
((E        )*(id+id))+id$ Reduce: E -> E+E

```

```

avyakths@AvysLegionSlim5:~ x + v
>> SUCCESS: The string "(id+id)*id" is VALID and ACCEPTED.
avyakths@AvysLegionSlim5:~/Compiler-Design-Lab/Lab_Assignment_3$ ./shift_reduce_parser
=====
EXPERIMENT 5: SHIFT-REDUCE PARSER
Grammar:
1. E -> E + E
2. E -> E * E
3. E -> (E)
4. E -> id
=====
Enter input string (e.g., id+id*id, (id+id), id*id): ((id+id)*(id+id))+id

STACK          INPUT BUFFER          ACTION
-----
(              (id+id)*(id+id))+id$  Shift (())
((             id+id)*(id+id))+id$  Shift (())
((id          +id)*(id+id))+id$     Shift (id)
((E          +id)*(id+id))+id$     Reduce: E -> id
((E+         id)*(id+id))+id$     Shift (+)
((E+id       )*(id+id))+id$     Shift (id)
((E+E       )*(id+id))+id$     Reduce: E -> id
((E         )*(id+id))+id$     Reduce: E -> E+E
((E)        *(id+id))+id$     Shift (())
(E          *(id+id))+id$     Reduce: E -> (E)
(E*         (id+id))+id$     Shift (*)
(E*(        id+id))+id$     Shift (())
(E*(id      +id))+id$     Shift (id)
(E*(E       +id))+id$     Reduce: E -> id
(E*(E+      id))+id$     Shift (+)
(E*(E+id    ))+id$     Shift (id)
(E*(E+E     ))+id$     Reduce: E -> id
(E*(E       ))+id$     Reduce: E -> E+E
(E*(E)      )+id$     Shift (())
(E*E        )+id$     Reduce: E -> (E)
(E          )+id$     Reduce: E -> E*E
(E)         +id$     Shift (())
E           +id$     Reduce: E -> (E)
E+          id$     Shift (+)
E+id        $      Shift (id)
E+E         $      Reduce: E -> id
E           $      Reduce: E -> E+E
E           $      ACCEPT

>> SUCCESS: The string "((id+id)*(id+id))+id" is VALID and ACCEPTED.
avyakths@AvysLegionSlim5:~/Compiler-Design-Lab/Lab_Assignment_3$ ./shift_reduce_parser
=====
EXPERIMENT 5: SHIFT-REDUCE PARSER
Grammar:
1. E -> E + E

```

```

avyakths@AvysLegionSlim5:~ x + v
avyakths@AvysLegionSlim5:~/Compiler-Design-Lab/Lab_Assignment_3$ ./shift_reduce_parser
=====
EXPERIMENT 5: SHIFT-REDUCE PARSER
Grammar:
1. E -> E + E
2. E -> E * E
3. E -> (E)
4. E -> id
=====
Enter input string (e.g., id+id*id, (id+id), id*id): id*id+id*id+id

STACK          INPUT BUFFER          ACTION
-----
id             *id+id*id+id$          Shift (id)
E              *id+id*id+id$          Reduce: E -> id
E*             id+id*id+id$          Shift (*)
E*id           +id*id+id$          Shift (id)
E*E            +id*id+id$          Reduce: E -> id
E              +id*id+id$          Reduce: E -> E*E
E+             id*id+id$          Shift (+)
E+id           *id+id$          Shift (id)
E+E            *id+id$          Reduce: E -> id
E              *id+id$          Reduce: E -> E+E
E*             id+id$          Shift (*)
E*id           +id$          Shift (id)
E*E            +id$          Reduce: E -> id
E              +id$          Reduce: E -> E*E
E+             id$          Shift (+)
E+id           $          Shift (id)
E+E            $          Reduce: E -> id
E              $          Reduce: E -> E+E
E              $          ACCEPT

```

```

>> SUCCESS: The string "id*id+id*id+id" is VALID and ACCEPTED.
avyakths@AvysLegionSlim5:~/Compiler-Design-Lab/Lab_Assignment_3$ ./shift_reduce_parser
=====
EXPERIMENT 5: SHIFT-REDUCE PARSER
Grammar:
1. E -> E + E
2. E -> E * E
3. E -> (E)
4. E -> id
=====
Enter input string (e.g., id+id*id, (id+id), id*id): id++id

STACK          INPUT BUFFER          ACTION
-----
id             ++id$          Shift (id)
E              ++id$          Reduce: E -> id
E+             +id$          Shift (+)

```

```

avyakths@AvysLegionSlim5:~ x + v
>> SUCCESS: The string "id*id+id*id+id" is VALID and ACCEPTED.
avyakths@AvysLegionSlim5:~/Compiler-Design-Lab/Lab_Assignment_3$ ./shift_reduce_parser
=====
EXPERIMENT 5: SHIFT-REDUCE PARSER
Grammar:
1. E -> E + E
2. E -> E * E
3. E -> (E)
4. E -> id
=====
Enter input string (e.g., id+id*id, (id+id), id*id): id++id

STACK          INPUT BUFFER          ACTION
-----
id             ++id$              Shift (id)
E              ++id$              Reduce: E -> id
E+             +id$              Shift (+)
E++            id$              Shift (+)
E++id          $              Shift (id)
E++E           $              Reduce: E -> id

>> ERROR: The string "id++id" is INVALID and REJECTED.
avyakths@AvysLegionSlim5:~/Compiler-Design-Lab/Lab_Assignment_3$ ./shift_reduce_parser
=====
EXPERIMENT 5: SHIFT-REDUCE PARSER
Grammar:
1. E -> E + E
2. E -> E * E
3. E -> (E)
4. E -> id
=====
Enter input string (e.g., id+id*id, (id+id), id*id): (id+id

STACK          INPUT BUFFER          ACTION
-----
(             id+id$              Shift ((
(id           +id$              Shift (id)
(E            +id$              Reduce: E -> id
(E+           id$              Shift (+)
(E+id         $              Shift (id)
(E+E          $              Reduce: E -> id
(E            $              Reduce: E -> E+E

>> ERROR: The string "(id+id" is INVALID and REJECTED.
avyakths@AvysLegionSlim5:~/Compiler-Design-Lab/Lab_Assignment_3$ ./shift_reduce_parser
=====
EXPERIMENT 5: SHIFT-REDUCE PARSER
Grammar:
1. E -> E + E

```

```

avyakths@AvysLegionSlim5:~ x + v
avyakths@AvysLegionSlim5:~/Compiler-Design-Lab/Lab_Assignment_3$ ./shift_reduce_parser
=====
EXPERIMENT 5: SHIFT-REDUCE PARSER
Grammar:
1. E -> E + E
2. E -> E * E
3. E -> (E)
4. E -> id
=====
Enter input string (e.g., id+id*id, (id+id), id*id): id**id

STACK          INPUT BUFFER      ACTION
-----
id             **id$             Shift (id)
E              **id$             Reduce: E -> id
E*             +id$             Shift (*)
E**            id$             Shift (+)
E**id          $             Shift (id)
E**E           $             Reduce: E -> id

>> ERROR: The string "id**id" is INVALID and REJECTED.
avyakths@AvysLegionSlim5:~/Compiler-Design-Lab/Lab_Assignment_3$ ./shift_reduce_parser
=====
EXPERIMENT 5: SHIFT-REDUCE PARSER
Grammar:
1. E -> E + E
2. E -> E * E
3. E -> (E)
4. E -> id
=====
Enter input string (e.g., id+id*id, (id+id), id*id): (

STACK          INPUT BUFFER      ACTION
-----
(              )$             Shift ((
(              $             Shift ())

>> ERROR: The string "( )" is INVALID and REJECTED.
avyakths@AvysLegionSlim5:~/Compiler-Design-Lab/Lab_Assignment_3$ ./shift_reduce_parser
=====
EXPERIMENT 5: SHIFT-REDUCE PARSER
Grammar:
1. E -> E + E
2. E -> E * E
3. E -> (E)
4. E -> id
=====
Enter input string (e.g., id+id*id, (id+id), id*id): +id*id

STACK          INPUT BUFFER      ACTION
-----

```

```

avyakths@AvysLegionSlim5:~ x + v
STACK          INPUT BUFFER          ACTION
-----
id             **id$                Shift (id)
E              **id$                Reduce: E -> id
E*             +id$                Shift (*)
E*+            id$                 Shift (+)
E*+id          $                  Shift (id)
E*+E           $                  Reduce: E -> id

>> ERROR: The string "id**id" is INVALID and REJECTED.
avyakths@AvysLegionSlim5:~/Compiler-Design-Lab/Lab_Assignment_3$ ./shift_reduce_parser
=====
EXPERIMENT 5: SHIFT-REDUCE PARSER
Grammar:
1. E -> E + E
2. E -> E * E
3. E -> (E)
4. E -> id
=====
Enter input string (e.g., id+id*id, (id+id), id*id): ( )

STACK          INPUT BUFFER          ACTION
-----
(              )$                  Shift (())
()             $                  Shift (())

>> ERROR: The string "()" is INVALID and REJECTED.
avyakths@AvysLegionSlim5:~/Compiler-Design-Lab/Lab_Assignment_3$ ./shift_reduce_parser
=====
EXPERIMENT 5: SHIFT-REDUCE PARSER
Grammar:
1. E -> E + E
2. E -> E * E
3. E -> (E)
4. E -> id
=====
Enter input string (e.g., id+id*id, (id+id), id*id): +id*id

STACK          INPUT BUFFER          ACTION
-----
+              id*id$              Shift (+)
+id            *id$                Shift (id)
+E            *id$                Reduce: E -> id
+E*           id$                 Shift (*)
+E*id         $                  Shift (id)
+E*E          $                  Reduce: E -> id
+E           $                  Reduce: E -> E*E

>> ERROR: The string "+id*id" is INVALID and REJECTED.
avyakths@AvysLegionSlim5:~/Compiler-Design-Lab/Lab_Assignment_3$ |

```

PROGRAM 2

lr0_parser.c

Aim:

To implement an LR(0) Parser in C by generating the canonical collection of LR(0) items, constructing the LR(0) Action and Goto parsing tables for a conflict-free grammar, and validating input strings through deterministic stack-based parsing.

About:

An LR(0) parser builds the canonical collection of LR(0) items using **Closure** and **Goto** operations. In an LR(0) table, a state with a completed production item ($A \rightarrow \alpha \cdot$) receives reduce actions across **all terminals** in that row.

Procedure:

1. **Augment the Grammar:** Given the grammar G , construct the augmented grammar G' by introducing a new start symbol and production $S' \rightarrow S$.
2. **Compute Canonical Collection of LR(0) Items:**
 - a. Define an item structure containing the LHS, RHS, and a dot position (dot).
 - b. Implement the CLOSURE(I) function: If item $[A \rightarrow \alpha \cdot B\beta]$ is in I , add $[B \rightarrow \cdot \gamma]$ for all productions of B until no new items can be added.

- c. Implement the GOTO(I, X) function: Transition the dot over symbol X for all items $[A \rightarrow \alpha \cdot X\beta]$ in I and compute the closure of the resulting set.
- d. Generate all unique states (sets of items) starting from $I_0 = \text{CLOSURE}(\{[S' \rightarrow \cdot S]\})$.

3. Construct the LR(0) Parsing Table:

- a. **Shift Actions:** If $[A \rightarrow \alpha \cdot a\beta] \in I_i$ and $\text{GOTO}(I_i, a) = I_j$, set $\text{ACTION}[i, a] = \text{shift } j$.
- b. **Goto Actions:** If $\text{GOTO}(I_i, A) = I_j$ for a non-terminal A , set $\text{GOTO}[i, A] = j$.
- c. **Reduce Actions:** If $[A \rightarrow \alpha \cdot] \in I_i$, set $\text{ACTION}[i, a] = \text{reduce } (A \rightarrow \alpha)$ for **all** terminals $a \in T \cup \{\$\}$.
- d. **Accept Action:** If $[S' \rightarrow S \cdot] \in I_i$, set $\text{ACTION}[i, \$] = \text{accept}$.

4. Parse the Input String:

- a. Initialize a state stack with state 0 and a symbol stack with \$.
- b. Look up the current state s (stack top) and the current input symbol a in $\text{ACTION}[s, a]$.
- c. If shift j : Push symbol a and state j onto their respective stacks, then advance the input pointer.
- d. If reduce $(A \rightarrow \alpha)$: Pop $|\alpha|$ symbols and states from the stacks. Look up state s' now on top of the stack and determine $j = \text{GOTO}[s', A]$. Push non-terminal A and state j .
- e. If accept: Output the trace table and conclude that the string is syntactically valid.
- f. If blank/error: Terminate and declare a syntax error.

C Program Code:

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

#define MAX_ITEMS 30
#define MAX_STATES 30
#define MAX_PROD 10

typedef struct {
    char lhs;
    char rhs[10];
    int dot;
} Item;

typedef struct {
    Item items[MAX_ITEMS];
    int count;
} State;

typedef struct {
    char lhs;
    char rhs[10];
} Production;

// Strictly LR(0) Prefix Expression Grammar
Production prods[MAX_PROD] = {
    {'E', "+EE"}, // 0
    {'E', "*EE"}, // 1
    {'E', "(E)"}, // 2
    {'E', "i"},   // 3
};

int num_prods = 4;

State states[MAX_STATES];
int num_states = 0;

char terminals[] = {'+', '*', '(', ')', 'i', '$'};
int num_terminals = 6;
char non_terminals[] = {'E'};
int num_non_terminals = 1;
```

```

char action_type[MAX_STATES][10];
int action_val[MAX_STATES][10];
int goto_table[MAX_STATES][10];

int item_exists(State *s, Item it) {
    for (int i = 0; i < s->count; i++) {
        if (s->items[i].lhs == it.lhs &&
            strcmp(s->items[i].rhs, it.rhs) == 0 &&
            s->items[i].dot == it.dot)
            return 1;
    }
    return 0;
}

void add_item(State *s, Item it) {
    if (!item_exists(s, it)) {
        s->items[s->count++] = it;
    }
}

void closure(State *s) {
    int added = 1;
    while (added) {
        added = 0;
        int current_count = s->count;
        for (int i = 0; i < current_count; i++) {
            Item it = s->items[i];
            if (it.dot < (int)strlen(it.rhs)) {
                char symbol = it.rhs[it.dot];
                for (int j = 0; j < num_prods; j++) {
                    if (prods[j].lhs == symbol) {
                        Item new_it = {prods[j].lhs, "", 0};
                        strcpy(new_it.rhs, prods[j].rhs);
                        if (!item_exists(s, new_it)) {
                            add_item(s, new_it);
                            added = 1;
                        }
                    }
                }
            }
        }
    }
}

```

```
}
```

```
State find_goto(State *s, char symbol) {  
    State res;  
    res.count = 0;  
    for (int i = 0; i < s->count; i++) {  
        Item it = s->items[i];  
        if (it.dot < (int)strlen(it.rhs) && it.rhs[it.dot] == symbol) {  
            Item next_it = it;  
            next_it.dot++;  
            add_item(&res, next_it);  
        }  
    }  
    if (res.count > 0) closure(&res);  
    return res;  
}
```

```
int find_state_index(State *s) {  
    for (int i = 0; i < num_states; i++) {  
        if (states[i].count == s->count) {  
            int match = 1;  
            for (int j = 0; j < s->count; j++) {  
                if (!item_exists(&states[i], s->items[j])) {  
                    match = 0;  
                    break;  
                }  
            }  
            if (match) return i;  
        }  
    }  
    return -1;  
}
```

```
int get_term_idx(char c) {  
    for (int i = 0; i < num_terminals; i++)  
        if (terminals[i] == c) return i;  
    return -1;  
}
```

```
int get_nterm_idx(char c) {  
    for (int i = 0; i < num_non_terminals; i++)  
        if (non_terminals[i] == c) return i;  
    return -1;  
}
```

```

}

void build_table() {
    State s0;
    s0.count = 0;
    Item init_item = {'Z', "E", 0}; // Z = E' (Augmented)
    add_item(&s0, init_item);
    closure(&s0);
    states[num_states++] = s0;

    for (int i = 0; i < num_states; i++) {
        // Shift transitions
        for (int t = 0; t < num_terminals - 1; t++) {
            char sym = terminals[t];
            State next_s = find_goto(&states[i], sym);
            if (next_s.count > 0) {
                int idx = find_state_index(&next_s);
                if (idx == -1) {
                    idx = num_states;
                    states[num_states++] = next_s;
                }
                action_type[i][t] = 's';
                action_val[i][t] = idx;
            }
        }

        // Goto transitions
        for (int nt = 0; nt < num_non_terminals; nt++) {
            char sym = non_terminals[nt];
            State next_s = find_goto(&states[i], sym);
            if (next_s.count > 0) {
                int idx = find_state_index(&next_s);
                if (idx == -1) {
                    idx = num_states;
                    states[num_states++] = next_s;
                }
                goto_table[i][nt] = idx;
            } else {
                goto_table[i][nt] = -1;
            }
        }

        // LR(0) Reduce and Accept transitions
    }
}

```

```

for (int j = 0; j < states[i].count; j++) {
    Item it = states[i].items[j];
    if (it.lhs == 'Z' && it.dot == (int)strlen(it.rhs)) {
        int dollar_idx = get_term_idx('$');
        action_type[i][dollar_idx] = 'a';
    } else if (it.dot == (int)strlen(it.rhs)) {
        int p_idx = -1;
        for (int k = 0; k < num_prods; k++) {
            if (prods[k].lhs == it.lhs && strcmp(prods[k].rhs, it.rhs) == 0) {
                p_idx = k;
                break;
            }
        }
        // LR(0): Place reduce across every terminal
        for (int t = 0; t < num_terminals; t++) {
            action_type[i][t] = 'r';
            action_val[i][t] = p_idx;
        }
    }
}
}
}

```

```

void print_table() {
    printf("\n===== LR(0) PARSING TABLE\n");
    printf("%-7s | %-35s | %-10s\n", "STATE", "ACTION", "GOTO");
    printf("    |");
    for (int i = 0; i < num_terminals; i++) printf(" %-5c", terminals[i]);
    printf(" |");
    for (int i = 0; i < num_non_terminals; i++) printf(" %-5c", non_terminals[i]);
    printf("\n-----\n");

    for (int i = 0; i < num_states; i++) {
        printf("I%-6d |", i);
        for (int t = 0; t < num_terminals; t++) {
            if (action_type[i][t] == 's') {
                char buf[10];
                snprintf(buf, sizeof(buf), "s%d", action_val[i][t]);
                printf(" %-5s", buf);
            } else if (action_type[i][t] == 'r') {
                char buf[10];
                snprintf(buf, sizeof(buf), "r%d", action_val[i][t]);
            }
        }
    }
}

```

```

        printf(" %-5s", buf);
    } else if (action_type[i][t] == 'a') {
        printf(" %-5s", "acc");
    } else {
        printf(" %-5s", ".");
    }
}
printf(" |");
for (int nt = 0; nt < num_non_terminals; nt++) {
    if (goto_table[i][nt] != -1) {
        printf(" %-5d", goto_table[i][nt]);
    } else {
        printf(" %-5s", ".");
    }
}
printf("\n");
}

```

```

printf("=====
=====\\n");
}

```

```

void parse_string(const char *raw_str) {
    char input[100];
    snprintf(input, sizeof(input), "%s$", raw_str);

    int state_stack[100];
    char sym_stack[100];
    int top = 0;

    state_stack[0] = 0;
    sym_stack[0] = '$';
    sym_stack[1] = '\\0';

    int ip = 0;

    printf("\\n%-25s %-15s %-20s %-20s\\n", "STATE STACK", "SYMBOL STACK",
"INPUT", "ACTION");
    printf("-----\\n");

    while (1) {
        int s = state_stack[top];
        char a = input[ip];
    }
}

```

```

int t_idx = get_term_idx(a);

if (t_idx == -1) {
    printf("\n>> ERROR: Invalid token '%c' in input stream.\n", a);
    return;
}

char state_str[50] = "";
for (int i = 0; i <= top; i++) {
    char buf[10];
    snprintf(buf, sizeof(buf), "%d ", state_stack[i]);
    strcat(state_str, buf);
}

char action_str[50] = "";
char act = action_type[s][t_idx];
int val = action_val[s][t_idx];

if (act == 's') {
    snprintf(action_str, sizeof(action_str), "Shift -> I%d", val);
    printf("%-25s %-15s %-20s %-20s\n", state_str, sym_stack, &input[ip],
action_str);
    top++;
    state_stack[top] = val;
    sym_stack[top] = a;
    sym_stack[top + 1] = '\0';
    ip++;
} else if (act == 'r') {
    snprintf(action_str, sizeof(action_str), "Reduce: %c -> %s", prods[val].lhs,
prods[val].rhs);
    printf("%-25s %-15s %-20s %-20s\n", state_str, sym_stack, &input[ip],
action_str);
    int len = strlen(prods[val].rhs);
    top -= len;
    int prev_state = state_stack[top];
    char lhs = prods[val].lhs;
    int nt_idx = get_nterm_idx(lhs);
    int next_state = goto_table[prev_state][nt_idx];

    top++;
    state_stack[top] = next_state;
    sym_stack[top] = lhs;
    sym_stack[top + 1] = '\0';
}

```

```

        } else if (act == 'a') {
            printf("%-25s %-15s %-20s %-20s\n", state_str, sym_stack, &input[ip],
"Accept");
            printf("\n>> SUCCESS: String \"%s\" successfully parsed and accepted!\n",
raw_str);
            return;
        } else {
            printf("%-25s %-15s %-20s %-20s\n", state_str, sym_stack, &input[ip],
"Error");
            printf("\n>> ERROR: Parsing failed. Syntax error at '%c'.\n", a);
            return;
        }
    }
}

int main() {

printf("===== \n");
printf("  EXPERIMENT 6: LR(0) PREFIX EXPRESSION PARSER  \n");
printf(" Grammar:                                \n");
printf("  0. E -> + E E                                \n");
printf("  1. E -> * E E                                \n");
printf("  2. E -> ( E )                                \n");
printf("  3. E -> i                                    \n");

printf("===== \n");

    build_table();
    print_table();

    char test_input[100];
    printf("\nEnter prefix expression to parse (e.g., +ii, *+iii, +(i)ii): ");
    if (scanf("%99s", test_input) == 1) {
        parse_string(test_input);
    }
    return 0;
}

```

Screenshots & Output:

```
avyakths@AvysLegionSlim5:~$ cd Compiler-Design-Lab/
avyakths@AvysLegionSlim5:~/Compiler-Design-Lab$ ls
Lab_Assignment_1 Lab_Assignment_2 Lab_Assignment_3
avyakths@AvysLegionSlim5:~/Compiler-Design-Lab$ cd Lab_Assignment_3
avyakths@AvysLegionSlim5:~/Compiler-Design-Lab/Lab_Assignment_3$ ls
lr0_parser lr0_parser.c shift_reduce_parser shift_reduce_parser.c
avyakths@AvysLegionSlim5:~/Compiler-Design-Lab/Lab_Assignment_3$ ./lr0_parser
=====
EXPERIMENT 6: LR(0) PREFIX EXPRESSION PARSER
Grammar:
0. E -> + E E
1. E -> * E E
2. E -> ( E )
3. E -> i
=====

===== LR(0) PARSING TABLE =====
STATE | ACTION | GOTO
      | +  *  (  )  i  $  | E
-----|-----|-----
I0    | s1    s2    s3    .    s4    .    | 5
I1    | s1    s2    s3    .    s4    .    | 6
I2    | s1    s2    s3    .    s4    .    | 7
I3    | s1    s2    s3    .    s4    .    | 8
I4    | r3    r3    r3    r3    r3    r3    | .
I5    | .      .      .      .      .      acc  | .
I6    | s1    s2    s3    .    s4    .    | 9
I7    | s1    s2    s3    .    s4    .    | 10
I8    | .      .      .      s11    .      .    | .
I9    | r0    r0    r0    r0    r0    r0    | .
I10   | r1    r1    r1    r1    r1    r1    | .
I11   | r2    r2    r2    r2    r2    r2    | .
=====

Enter prefix expression to parse (e.g., +ii, ++iii, +(i)ii): i

STATE STACK      SYMBOL STACK      INPUT      ACTION
-----
0              $              i$          Shift -> I4
0 4             $i             $           Reduce: E -> i
0 5             $E             $           Accept

>> SUCCESS: String "i" successfully parsed and accepted!
avyakths@AvysLegionSlim5:~/Compiler-Design-Lab/Lab_Assignment_3$ ./lr0_parser
=====
EXPERIMENT 6: LR(0) PREFIX EXPRESSION PARSER
Grammar:
0. E -> + E E
1. E -> * E E
2. E -> ( E )
```

```

0 5          $E          $          Accept

>> SUCCESS: String "i" successfully parsed and accepted!
avyakths@AvysLegionSlim5:~/Compiler-Design-Lab/Lab_Assignment_3$ ./lr0_parser
=====

```

EXPERIMENT 6: LR(0) PREFIX EXPRESSION PARSER

Grammar:

- 0. E -> + E E
- 1. E -> * E E
- 2. E -> (E)
- 3. E -> i

=====

LR(0) PARSING TABLE =====

STATE	ACTION						GOTO
	+	*	(	)	i	\$	
I0	s1	s2	s3	.	s4	.	5
I1	s1	s2	s3	.	s4	.	6
I2	s1	s2	s3	.	s4	.	7
I3	s1	s2	s3	.	s4	.	8
I4	r3	r3	r3	r3	r3	r3	.
I5	.	.	.	.	.	acc	.
I6	s1	s2	s3	.	s4	.	9
I7	s1	s2	s3	.	s4	.	10
I8	.	.	.	s11	.	.	.
I9	r0	r0	r0	r0	r0	r0	.
I10	r1	r1	r1	r1	r1	r1	.
I11	r2	r2	r2	r2	r2	r2	.

=====

Enter prefix expression to parse (e.g., +ii, **iii, +(i)ii): +ii

STATE	STACK	SYMBOL	STACK	INPUT	ACTION
0		\$		+ii\$	Shift -> I1
0 1		\$+		ii\$	Shift -> I4
0 1 4		\$+i		i\$	Reduce: E -> i
0 1 6		\$+E		i\$	Shift -> I4
0 1 6 4		\$+Ei		\$	Reduce: E -> i
0 1 6 9		\$+EE		\$	Reduce: E -> +EE
0 5		\$E		\$	Accept

```

>> SUCCESS: String "+ii" successfully parsed and accepted!
avyakths@AvysLegionSlim5:~/Compiler-Design-Lab/Lab_Assignment_3$ ./lr0_parser
=====

```

EXPERIMENT 6: LR(0) PREFIX EXPRESSION PARSER

Grammar:

- 0. E -> + E E
- 1. E -> * E E

```

avyakths@AvysLegionSlim5:~ × + v
0 1 6 9      $+EE      $      Reduce: E -> +EE
0 5          $E        $      Accept

```

```

>> SUCCESS: String "+ii" successfully parsed and accepted!
avyakths@AvysLegionSlim5:~/Compiler-Design-Lab/Lab_Assignment_3$ ./lr0_parser

```

EXPERIMENT 6: LR(0) PREFIX EXPRESSION PARSER

Grammar:

0. E -> + E E
1. E -> * E E
2. E -> (E)
3. E -> i

===== LR(0) PARSING TABLE =====

STATE	ACTION						GOTO
	+	*	(	)	i	\$	
I0	s1	s2	s3	.	s4	.	5
I1	s1	s2	s3	.	s4	.	6
I2	s1	s2	s3	.	s4	.	7
I3	s1	s2	s3	.	s4	.	8
I4	r3	r3	r3	r3	r3	r3	.
I5	.	.	.	.	.	acc	.
I6	s1	s2	s3	.	s4	.	9
I7	s1	s2	s3	.	s4	.	10
I8	.	.	.	s11	.	.	.
I9	r0	r0	r0	r0	r0	r0	.
I10	r1	r1	r1	r1	r1	r1	.
I11	r2	r2	r2	r2	r2	r2	.

=====

Enter prefix expression to parse (e.g., +ii, ++iii, +(i)ii): ++iii

STATE	STACK	SYMBOL STACK	INPUT	ACTION
0		\$	*+iii\$	Shift -> I2
0 2		\$*	+iii\$	Shift -> I1
0 2 1		\$*+	iii\$	Shift -> I4
0 2 1 4		\$*+i	ii\$	Reduce: E -> i
0 2 1 6		\$*+E	ii\$	Shift -> I4
0 2 1 6 4		\$*+Ei	i\$	Reduce: E -> i
0 2 1 6 9		\$*+EE	i\$	Reduce: E -> +EE
0 2 7		\$*E	i\$	Shift -> I4
0 2 7 4		\$*Ei	\$	Reduce: E -> i
0 2 7 10		\$*EE	\$	Reduce: E -> *EE
0 5		\$E	\$	Accept

```

>> SUCCESS: String "++iii" successfully parsed and accepted!
avyakths@AvysLegionSlim5:~/Compiler-Design-Lab/Lab_Assignment_3$ ./lr0_parser

```

```

avyakths@AvysLegionSlim5:~ × + v
avyakths@AvysLegionSlim5:~/Compiler-Design-Lab/Lab_Assignment_3$ ./lr0_parser

```

EXPERIMENT 6: LR(0) PREFIX EXPRESSION PARSER

Grammar:

0. $E \rightarrow + E E$
1. $E \rightarrow * E E$
2. $E \rightarrow (E)$
3. $E \rightarrow i$

===== LR(0) PARSING TABLE =====

STATE	ACTION						GOTO
	+	*	(	)	i	\$	
I0	s1	s2	s3	.	s4	.	5
I1	s1	s2	s3	.	s4	.	6
I2	s1	s2	s3	.	s4	.	7
I3	s1	s2	s3	.	s4	.	8
I4	r3	r3	r3	r3	r3	r3	.
I5	.	.	.	.	.	acc	.
I6	s1	s2	s3	.	s4	.	9
I7	s1	s2	s3	.	s4	.	10
I8	.	.	.	s11	.	.	.
I9	r0	r0	r0	r0	r0	r0	.
I10	r1	r1	r1	r1	r1	r1	.
I11	r2	r2	r2	r2	r2	r2	.

=====

Enter prefix expression to parse (e.g., +ii, ++iii, +(i)ii): ++ii+ii

STATE	STACK	SYMBOL	STACK	INPUT	ACTION
0		\$		++ii+ii\$	Shift -> I1
0 1		\$+		++ii+ii\$	Shift -> I2
0 1 2		\$++		++ii+ii\$	Shift -> I4
0 1 2 4		\$++i		++ii+ii\$	Reduce: E -> i
0 1 2 7		\$++E		++ii+ii\$	Shift -> I4
0 1 2 7 4		\$++Ei		++ii\$	Reduce: E -> i
0 1 2 7 10		\$++EE		++ii\$	Reduce: E -> *EE
0 1 6		\$+E		++ii\$	Shift -> I1
0 1 6 1		\$+E+		++ii\$	Shift -> I4
0 1 6 1 4		\$+E+i		++i\$	Reduce: E -> i
0 1 6 1 6		\$+E+E		++i\$	Shift -> I4
0 1 6 1 6 4		\$+E+Ei		++\$	Reduce: E -> i
0 1 6 1 6 9		\$+E+EE		++\$	Reduce: E -> +EE
0 1 6 9		\$+EE		++\$	Reduce: E -> +EE
0 5		\$E		++\$	Accept

>> SUCCESS: String "+ii+ii" successfully parsed and accepted!

```

avyakths@AvysLegionSlim5:~/Compiler-Design-Lab/Lab_Assignment_3$ ./lr0_parser

```

avyakths@AvysLegionSlim5:~

```
>> SUCCESS: String "+*ii+ii" successfully parsed and accepted!
avyakths@AvysLegionSlim5:~/Compiler-Design-Lab/Lab_Assignment_3$ ./lr0_parser
```

EXPERIMENT 6: LR(0) PREFIX EXPRESSION PARSER

Grammar:

- 0. $E \rightarrow + E E$
- 1. $E \rightarrow * E E$
- 2. $E \rightarrow (E)$
- 3. $E \rightarrow i$

LR(0) PARSING TABLE							
STATE	ACTION						GOTO
	+	*	(	)	i	\$	
I0	s1	s2	s3	.	s4	.	5
I1	s1	s2	s3	.	s4	.	6
I2	s1	s2	s3	.	s4	.	7
I3	s1	s2	s3	.	s4	.	8
I4	r3	r3	r3	r3	r3	r3	.
I5	.	.	.	.	.	acc	.
I6	s1	s2	s3	.	s4	.	9
I7	s1	s2	s3	.	s4	.	10
I8	.	.	.	s11	.	.	.
I9	r0	r0	r0	r0	r0	r0	.
I10	r1	r1	r1	r1	r1	r1	.
I11	r2	r2	r2	r2	r2	r2	.

Enter prefix expression to parse (e.g., +ii, +*iii, +*(i)ii): +*(i)i+i(i)

STATE STACK	SYMBOL STACK	INPUT	ACTION
0	\$	+(i)i+i(i)\$	Shift -> I1
0 1	\$+	*(i)i+i(i)\$	Shift -> I2
0 1 2	\$**	(i)i+i(i)\$	Shift -> I3
0 1 2 3	\$+*(	i)i+i(i)\$	Shift -> I4
0 1 2 3 4	\$+*(i	)i+i(i)\$	Reduce: $E \rightarrow i$
0 1 2 3 8	\$+*(E	)i+i(i)\$	Shift -> I11
0 1 2 3 8 11	\$+*(E)	i+i(i)\$	Reduce: $E \rightarrow (E)$
0 1 2 7	\$+*E	i+i(i)\$	Shift -> I4
0 1 2 7 4	\$+*Ei	+i(i)\$	Reduce: $E \rightarrow i$
0 1 2 7 10	\$+*EE	+i(i)\$	Reduce: $E \rightarrow *EE$
0 1 6	\$+E	+i(i)\$	Shift -> I1
0 1 6 1	\$+E+	i(i)\$	Shift -> I4
0 1 6 1 4	\$+E+i	(i)\$	Reduce: $E \rightarrow i$
0 1 6 1 6	\$+E+E	(i)\$	Shift -> I3
0 1 6 1 6 3	\$+E+E(	i)\$	Shift -> I4
0 1 6 1 6 3 4	\$+E+E(i	)\$	Reduce: $E \rightarrow i$
0 1 6 1 6 3 8	\$+E+E(E	)\$	Shift -> I11
0 1 6 1 6 3 8 11	\$+E+E(E)	\$	Reduce: $E \rightarrow (E)$
0 1 6 1 6 9	\$+E+EE	\$	Reduce: $E \rightarrow +EE$
0 1 6 9	\$+EE	\$	Reduce: $E \rightarrow +EE$
0 5	\$E	\$	Accept

```
>> SUCCESS: String "+*(i)i+i(i)" successfully parsed and accepted!
avyakths@AvysLegionSlim5:~/Compiler-Design-Lab/Lab_Assignment_3$ ./lr0_parser
```

```
>> SUCCESS: String "+*(i)ii+i(i)" successfully parsed and accepted!
avyakths@AvysLegionSlim5:~/Compiler-Design-Lab/Lab_Assignment_3$ ./lr0_parser
```

EXPERIMENT 6: LR(0) PREFIX EXPRESSION PARSER

Grammar:

0. E -> + E E
1. E -> * E E
2. E -> (E)
3. E -> i

LR(0) PARSING TABLE							
STATE	ACTION						GOTO
	+	*	(	)	i	\$	
I0	s1	s2	s3	.	s4	.	5
I1	s1	s2	s3	.	s4	.	6
I2	s1	s2	s3	.	s4	.	7
I3	s1	s2	s3	.	s4	.	8
I4	r3	r3	r3	r3	r3	r3	.
I5	.	.	.	.	.	acc	.
I6	s1	s2	s3	.	s4	.	9
I7	s1	s2	s3	.	s4	.	10
I8	.	.	.	s11	.	.	.
I9	r0	r0	r0	r0	r0	r0	.
I10	r1	r1	r1	r1	r1	r1	.
I11	r2	r2	r2	r2	r2	r2	.

Enter prefix expression to parse (e.g., +ii, +iii, +(i)ii): +i*ii+(i)*ii

STATE	STACK	SYMBOL STACK	INPUT	ACTION
0		\$	+i*ii+(i)*ii\$	Shift -> I2
0 2		\$*	+i*ii+(i)*ii\$	Shift -> I1
0 2 1		\$**	i*ii+(i)*ii\$	Shift -> I4
0 2 1 4		\$**i	*ii+(i)*ii\$	Reduce: E -> i
0 2 1 6		\$**E	*ii+(i)*ii\$	Shift -> I2
0 2 1 6 2		\$**E*	ii+(i)*ii\$	Shift -> I4
0 2 1 6 2 4		\$**E*i	i+(i)*ii\$	Reduce: E -> i
0 2 1 6 2 7		\$**E*E	i+(i)*ii\$	Shift -> I4
0 2 1 6 2 7 4		\$**E*Ei	+(i)*ii\$	Reduce: E -> i
0 2 1 6 2 7 10		\$**E*EE	+(i)*ii\$	Reduce: E -> *EE
0 2 1 6 9		\$**EE	+(i)*ii\$	Reduce: E -> +EE
0 2 7		\$*E	+(i)*ii\$	Shift -> I1
0 2 7 1		\$*E+	(i)*ii\$	Shift -> I3
0 2 7 1 3		\$*E+(	i)*ii\$	Shift -> I4
0 2 7 1 3 4		\$*E+(i	)*ii\$	Reduce: E -> i
0 2 7 1 3 8		\$*E+(E	)*ii\$	Shift -> I11
0 2 7 1 3 8 11		\$*E+(E)	*ii\$	Reduce: E -> (E)
0 2 7 1 6		\$*E+E	*ii\$	Shift -> I2
0 2 7 1 6 2		\$*E+E*	ii\$	Shift -> I4
0 2 7 1 6 2 4		\$*E+E*i	i\$	Reduce: E -> i
0 2 7 1 6 2 7		\$*E+E*E	i\$	Shift -> I4
0 2 7 1 6 2 7 4		\$*E+E*Ei	\$	Reduce: E -> i
0 2 7 1 6 2 7 10		\$*E+E*EE	\$	Reduce: E -> *EE
0 2 7 1 6 9		\$*E+EE	\$	Reduce: E -> +EE
0 2 7 10		\$*EE	\$	Reduce: E -> *EE
0 5		\$E	\$	Accept

```
>> SUCCESS: String "+i*ii+(i)*ii" successfully parsed and accepted!
avyakths@AvysLegionSlim5:~/Compiler-Design-Lab/Lab_Assignment_3$ ./lr0_parser
```

EXPERIMENT 6: LR(0) PREFIX EXPRESSION PARSER

Grammar:

```

avyakths@AvysLegionSlim5:~ × + v
avyakths@AvysLegionSlim5:~/Compiler-Design-Lab/Lab_Assignment_3$ ./lr0_parser
=====

```

EXPERIMENT 6: LR(0) PREFIX EXPRESSION PARSER

Grammar:

0. E -> + E E
1. E -> * E E
2. E -> (E)
3. E -> i

=====

===== LR(0) PARSING TABLE =====

STATE	ACTION						GOTO
	+	*	(	)	i	\$	E
I0	s1	s2	s3	.	s4	.	5
I1	s1	s2	s3	.	s4	.	6
I2	s1	s2	s3	.	s4	.	7
I3	s1	s2	s3	.	s4	.	8
I4	r3	r3	r3	r3	r3	r3	.
I5	.	.	.	.	.	acc	.
I6	s1	s2	s3	.	s4	.	9
I7	s1	s2	s3	.	s4	.	10
I8	.	.	.	s11	.	.	.
I9	r0	r0	r0	r0	r0	r0	.
I10	r1	r1	r1	r1	r1	r1	.
I11	r2	r2	r2	r2	r2	r2	.

=====

Enter prefix expression to parse (e.g., +ii, **iii, +(i)ii): +i

STATE	STACK	SYMBOL STACK	INPUT	ACTION
0		\$	+i\$	Shift -> I1
0 1		\$+	i\$	Shift -> I4
0 1 4		\$+i	\$	Reduce: E -> i
0 1 6		\$+E	\$	Error

>> ERROR: Parsing failed. Syntax error at '\$'.

```

avyakths@AvysLegionSlim5:~/Compiler-Design-Lab/Lab_Assignment_3$ ./lr0_parser
=====

```

EXPERIMENT 6: LR(0) PREFIX EXPRESSION PARSER

Grammar:

0. E -> + E E
1. E -> * E E
2. E -> (E)
3. E -> i

=====

===== LR(0) PARSING TABLE =====

STATE	ACTION						GOTO
	+	*	(	)	i	\$	E
I0	s1	s2	s3	.	s4	.	5
I1	s1	s2	s3	.	s4	.	6



avyakths@AvysLegionSlim5:~



avyakths@AvysLegionSlim5:~/Compiler-Design-Lab/Lab_Assignment_3\$./lr0_parser

=====

EXPERIMENT 6: LR(0) PREFIX EXPRESSION PARSER

Grammar:

- 0. E -> + E E
- 1. E -> * E E
- 2. E -> (E)
- 3. E -> i

===== LR(0) PARSING TABLE =====							
STATE	ACTION						GOTO
	+	*	(	)	i	\$	E
I0	s1	s2	s3	.	s4	.	5
I1	s1	s2	s3	.	s4	.	6
I2	s1	s2	s3	.	s4	.	7
I3	s1	s2	s3	.	s4	.	8
I4	r3	r3	r3	r3	r3	r3	.
I5	.	.	.	.	.	acc	.
I6	s1	s2	s3	.	s4	.	9
I7	s1	s2	s3	.	s4	.	10
I8	.	.	.	s11	.	.	.
I9	r0	r0	r0	r0	r0	r0	.
I10	r1	r1	r1	r1	r1	r1	.
I11	r2	r2	r2	r2	r2	r2	.

Enter prefix expression to parse (e.g., +ii, *+iii, +(i)ii): i+i

STATE	STACK	SYMBOL STACK	INPUT	ACTION
0		\$	i+i\$	Shift -> I4
0 4		\$i	+i\$	Reduce: E -> i
0 5		\$E	+i\$	Error

>> ERROR: Parsing failed. Syntax error at '+'.
=====

avyakths@AvysLegionSlim5:~/Compiler-Design-Lab/Lab_Assignment_3\$./lr0_parser

=====

EXPERIMENT 6: LR(0) PREFIX EXPRESSION PARSER

Grammar:

- 0. E -> + E E
- 1. E -> * E E
- 2. E -> (E)
- 3. E -> i

===== LR(0) PARSING TABLE =====							
STATE	ACTION						GOTO
	+	*	(	)	i	\$	E
I0	s1	s2	s3	.	s4	.	5
I1	s1	s2	s3	.	s4	.	6
I2	s1	s2	s3	.	s4	.	7
I3	s1	s2	s3	.	s4	.	8
I4	r3	r3	r3	r3	r3	r3	.
I5	.	.	.	.	.	acc	.
I6	s1	s2	s3	.	s4	.	9
I7	s1	s2	s3	.	s4	.	10
I8	.	.	.	s11	.	.	.
I9	r0	r0	r0	r0	r0	r0	.
I10	r1	r1	r1	r1	r1	r1	.
I11	r2	r2	r2	r2	r2	r2	.

Enter prefix expression to parse (e.g., +ii, *+iii, +(i)ii): +ii i

STATE	STACK	SYMBOL STACK	INPUT	ACTION
0		\$	+ii\$	Shift -> I1
0 1		\$+	ii\$	Shift -> I4
0 1 4		\$+i	i\$	Reduce: E -> i
0 1 6		\$+E	i\$	Shift -> I4
0 1 6 4		\$+Ei	\$	Reduce: E -> i
0 1 6 9		\$+EE	\$	Reduce: E -> +EE
0 5		\$E	\$	Accept

>> SUCCESS: String "+ii" successfully parsed and accepted!

avyakths@AvysLegionSlim5:~/Compiler-Design-Lab/Lab_Assignment_3\$./lr0_parser

=====

```

avyakths@AvysLegionSlim5:~ × + v
>> SUCCESS: String "+ii" successfully parsed and accepted!
avyakths@AvysLegionSlim5:~/Compiler-Design-Lab/Lab_Assignment_3$ ./lr0_parser
=====

```

EXPERIMENT 6: LR(0) PREFIX EXPRESSION PARSER

Grammar:

0. E -> + E E
1. E -> * E E
2. E -> (E)
3. E -> i

===== LR(0) PARSING TABLE =====

STATE	ACTION						GOTO
	+	*	(	)	i	\$	
I0	s1	s2	s3	.	s4	.	5
I1	s1	s2	s3	.	s4	.	6
I2	s1	s2	s3	.	s4	.	7
I3	s1	s2	s3	.	s4	.	8
I4	r3	r3	r3	r3	r3	r3	.
I5	.	.	.	.	.	acc	.
I6	s1	s2	s3	.	s4	.	9
I7	s1	s2	s3	.	s4	.	10
I8	.	.	.	s11	.	.	.
I9	r0	r0	r0	r0	r0	r0	.
I10	r1	r1	r1	r1	r1	r1	.
I11	r2	r2	r2	r2	r2	r2	.

=====

Enter prefix expression to parse (e.g., +ii, *+iii, *(i)ii): *(i))i

STATE	STACK	SYMBOL STACK	INPUT	ACTION
0		\$	*(i))i\$	Shift -> I2
0 2		\$\$	(i))i\$	Shift -> I3
0 2 3		\$(	i))i\$	Shift -> I4
0 2 3 4		\$(i	)i\$	Reduce: E -> i
0 2 3 8		\$(E	)i\$	Shift -> I11
0 2 3 8 11		\$(E)	)i\$	Reduce: E -> (E)
0 2 7		\$\$E	)i\$	Error

```

>> ERROR: Parsing failed. Syntax error at ')'.
avyakths@AvysLegionSlim5:~/Compiler-Design-Lab/Lab_Assignment_3$ ./lr0_parser
=====

```

EXPERIMENT 6: LR(0) PREFIX EXPRESSION PARSER

Grammar:

0. E -> + E E
1. E -> * E E
2. E -> (E)
3. E -> i

===== LR(0) PARSING TABLE =====

```

avyakths@AvysLegionSlim5:~ × + v
STATE STACK          SYMBOL STACK    INPUT          ACTION
-----
0                   $                *(i))i$        Shift -> I2
0 2                 $*                (i))i$        Shift -> I3
0 2 3               $*(              i))i$        Shift -> I4
0 2 3 4             $*(i            ))i$        Reduce: E -> i
0 2 3 8             $*(E            ))i$        Shift -> I11
0 2 3 8 11          $*(E)           )i$        Reduce: E -> (E)
0 2 7               $*E              )i$        Error

>> ERROR: Parsing failed. Syntax error at ')'.
avyakths@AvysLegionSlim5:~/Compiler-Design-Lab/Lab_Assignment_3$ ./lr0_parser
=====
EXPERIMENT 6: LR(0) PREFIX EXPRESSION PARSER
Grammar:
0. E -> + E E
1. E -> * E E
2. E -> ( E )
3. E -> i
=====

===== LR(0) PARSING TABLE =====
STATE | ACTION |          |          |          |          |          | GOTO
      | +      | *      | (      | )      | i      | $      | E
-----|-----|-----|-----|-----|-----|-----|-----
I0     | s1     | s2     | s3     | .      | s4     | .      | 5
I1     | s1     | s2     | s3     | .      | s4     | .      | 6
I2     | s1     | s2     | s3     | .      | s4     | .      | 7
I3     | s1     | s2     | s3     | .      | s4     | .      | 8
I4     | r3     | r3     | r3     | r3     | r3     | r3     | .
I5     | .      | .      | .      | .      | .      | acc    | .
I6     | s1     | s2     | s3     | .      | s4     | .      | 9
I7     | s1     | s2     | s3     | .      | s4     | .      | 10
I8     | .      | .      | .      | s11    | .      | .      | .
I9     | r0     | r0     | r0     | r0     | r0     | r0     | .
I10    | r1     | r1     | r1     | r1     | r1     | r1     | .
I11    | r2     | r2     | r2     | r2     | r2     | r2     | .
=====

Enter prefix expression to parse (e.g., +ii, **iii, +(i)ii): **ii

STATE STACK          SYMBOL STACK    INPUT          ACTION
-----
0                   $                **ii$         Shift -> I1
0 1                 $+                **ii$         Shift -> I2
0 1 2               $**                ii$          Shift -> I4
0 1 2 4             $**i              i$           Reduce: E -> i
0 1 2 7             $**E              i$           Shift -> I4
0 1 2 7 4           $**Ei              $            Reduce: E -> i
0 1 2 7 10          $**EE              $            Reduce: E -> *EE
0 1 6               $+E              $            Error

>> ERROR: Parsing failed. Syntax error at '$'.
avyakths@AvysLegionSlim5:~/Compiler-Design-Lab/Lab_Assignment_3$ |

```

THANK YOU